

Integration Patterns Exercise

Oct 9, 2026 · @Antonio

How to run it

All figures are hypothetical. For a quick vote in the Teams chat, the nine patterns are numbered as on the slide:

1. Peer-to-Peer
2. API Gateway
3. Pub-Sub
4. Request-Response
5. Event Sourcing
6. ETL
7. Batching
8. Streaming Processing
9. Orchestration

Scenario 1: Who needs to know?

Our hypothetical Loan Application System records each new application and returns "Pending assessment". Four other teams need to react when an application is received:

- **Customer Communication** sends a confirmation email and SMS.
- **The CRM** (customer relationship management system) adds the event to the customer timeline that branch advisers see.
- **Risk Reporting** counts applications by product and region.
- **Marketing** removes the customer from the running "pre-approved loan" campaign.

Next quarter, Fraud Monitoring and Data Science want the same information. The loan team does not want to change and redeploy its system every time a new team joins. If the CRM is unavailable for an hour, application intake must not slow down or fail.

Which of the nine patterns should connect the Loan Application System to these teams? Which pattern is the tempting wrong choice, and why?

Scenario 2: Six steps, three weeks

A home-mortgage request moves through six systems, always in this order:

1. **Identity and KYC (know-your-customer) check:** Onboarding.
2. **Credit decision:** Credit Decision Service.
3. **Property valuation** by an external surveyor: takes 3 to 10 days.
4. **Contract generation:** Document Service.
5. **Customer e-signature:** may take up to 30 days.
6. **Disbursement:** Core Banking.

In addition:

- if the valuation is lower than the amount requested, the credit decision must run again with the new value;
- if the customer has not signed after 30 days, the reserved funds must be released and the case closed;
- Operations wants to see, for every case, the current step and how long it has been waiting there;
- internal audit needs evidence of what happened at each step, and when.

Which pattern should run this process end to end? What would go wrong if each system simply called, or notified, the next one?

Scenario 3: Two countries in ten minutes

The bank's 6 million cards generate up to 1,500 payment authorisations per second. Today a nightly job analyses the day's transactions and flags suspicious cards the next morning, often after the money is gone.

The business wants to catch patterns such as:

- the same card used in two countries within 10 minutes;
- more than five small online payments within two minutes ("card testing": fraudsters checking whether stolen card numbers work);
- spending ten times the customer's usual daily amount.

When one of these patterns appears, the card must be blocked and the customer alerted within 5 seconds. Each payment, taken on its own, may look perfectly normal.

Which integration pattern should detect these fraud patterns? Which pattern does today's nightly job use, and why does it fail here?

Bonus for fast groups

In Scenario 2, audit now asks to reconstruct any case exactly as it was on any past date.

Which pattern adds that capability?

Facilitator key (not for participants)

Scenario 1 → 3. Pub-Sub

Why. The Loan Application System publishes one fact ("ApplicationReceived") to a topic, and each team subscribes to it. New teams join without touching the producer, and a CRM outage only delays the CRM's own subscription: intake is unaffected.

Tempting wrong choices.

- **Peer-to-Peer:** six direct calls; every new team means a change and a redeploy, and a CRM outage reaches intake.
- **Orchestration:** there is no process to coordinate; the teams act independently.

Twist to discuss.

- What goes in the event: Marketing should not receive more personal data than it needs, so we publish a reference and minimal data.
- Reliable publishing: the transactional outbox (Chapter 02).
- Patterns coexist: the daily risk dashboard could be fed by ETL or Batching.

Scenario 2 → 9. Orchestration

Why. One orchestrator (a workflow engine) owns the step order, each case's state, the timers (the 30-day signature), the loop back to the credit decision, the compensation (releasing the reserved funds) and the audit trail.

What goes wrong otherwise.

- **Chained Request-Response:** no call can wait 10 days for a surveyor or 30 days for a signature.
- **Pub-Sub alone** ("choreography": each system reacting to the previous one's event): the flow exists only as scattered reactions, nobody can answer "where is case M-2210, and since when?", and timers and compensations are spread across six systems.

Twist to discuss.

- The orchestrator decides the flow, not the transport: its calls can be request-response or messages.
- It coordinates, but must not absorb credit policy, which stays in the Credit Decision Service.
- Link to the saga and compensation slide in Chapter 02.

Scenario 3 → 8. Streaming Processing

Why. Authorisations arrive as a continuous flow; a stream processor keeps state per card over time windows (10 minutes, 2 minutes, the usual daily amount) and reacts within seconds.

Other candidates.

- **Batching** is today's approach: fine for many reports, but hours too late here.
- **ETL** feeds the data warehouse: good for monthly fraud analysis, not for a 5-second reaction.
- **Request-Response** scoring of each payment in isolation cannot see "this card was in Milan six minutes ago" unless it keeps state across events, which is exactly what streaming does.

Twist to discuss.

- Banks usually combine the two: request-response scoring inside the authorisation (a budget of milliseconds) and streaming for patterns that span many events.
- Detection triggers actions (block the card, alert the customer, open a case), often published via Pub-Sub.

Bonus → 5. Event Sourcing

Storing every change to the case as an event makes it possible to rebuild its state at any past date.